CLAUDE CERTIFICATION GUIDE · OCTOBER 2026 EDITION

CCA-F

Claude Certified Architect - Foundations

The complete study guide, with 30 practice questions and an answer key

Full-length timed practice tests: preporato.com/certificates/claude-certified-architect

Updates, the practice test and the other three guides: github.com/preporato/claude-certification-guide

MIT license. Maintained by Preporato. Not affiliated with, endorsed by, or sponsored by Anthropic; Claude is a trademark of Anthropic, PBC. Exam facts reflect the program in October 2026; the Anthropic Partner Academy is authoritative.

Claude Certified Architect - Foundations (CCA-F): The Complete Study Guide

A full preparation guide for Anthropic's CCA-F exam. Part I teaches the technology the exam draws from, organized by topic. Part II re-indexes the same material by exam domain for revision. Part III covers exam strategy, worked questions, and a pre-exam checklist.

Maintained by preporato.com. Facts checked against the official exam guide and Anthropic's program pages in October 2026.

Contents

- Introduction
- Exam format
- The six production scenarios
- **Part I: Technology**
  - Chapter 1: The Claude API and the agentic loop
  - Chapter 2: Tools and tool_use
  - Chapter 3: Multi-agent orchestration
  - Chapter 4: Model Context Protocol (MCP)
  - Chapter 5: Claude Code
  - Chapter 6: Prompt engineering and structured output
  - Chapter 7: Context management in production
  - Chapter 8: Reliability, escalation, and recovery
- **Part II: Exam domain notes**
  - Domain 1: Agentic Architecture & Orchestration (27%)
  - Domain 2: Tool Design & MCP Integration (18%)
  - Domain 3: Claude Code Configuration & Workflows (20%)
  - Domain 4: Prompt Engineering & Structured Output (20%)
  - Domain 5: Context Management & Reliability (15%)
- **Part III: Exam preparation**
  - The preparation ladder
  - The Golden Rule and five mental models
  - The seven anti-patterns
  - Critical distinctions
  - Exam-day strategy
  - Worked questions
  - 30-day study plan
  - Pre-exam checklist
  - Resources

Introduction

CCA-F (Claude Certified Architect - Foundations) is Anthropic's certification for people who build production-grade Claude systems end to end: agentic architectures, multi-agent orchestration, prompt and context engineering for reliability, and the tools and MCP servers agents depend on. It launched on March 12, 2026 as the program's first exam and now sits inside a four-certification program (see the repo root for the full comparison).

The exam is written from production judgment. Questions describe running systems with logs, failures, and constraints, and ask what a competent architect would do. Memorizing definitions is necessary but never sufficient; the exam repeatedly asks you to choose between two plausible approaches, and the difference is always an engineering principle this guide teaches explicitly.

Who should take it: engineers and technical leads who have moved past making individual API calls and now design the systems those calls live in. Anthropic recommends hands-on experience building Claude or agent systems before sitting it.

Exam format

Questions	60 multiple-choice, all scored
Duration	120 minutes to answer, about 135 minutes of seat time
Passing score	720 on a 100-1000 scaled score
Question style	Single-answer plus multiple-response ("Select TWO/THREE", roughly a quarter of questions)
Structure	Scenario-anchored: 4 of 6 production scenarios per sitting, plus standalone questions
Delivery	Pearson VUE, test center or online proctored. No Claude, no docs, no browser during the exam
Price	$125 per attempt
Registration	Anthropic Partner Academy, open to people who work at Claude Partner Network organizations (sign in with a company email)
Languages	English or Japanese
Retakes	Full fee for every attempt; waits of 14, 30 and 90 days after the first, second and third failed attempts; at most 4 attempts in any 12 months
Validity	1 year; free non-proctored renewal assessment before expiration, full-price proctored retake after it lapses

Exam domains

Domain	Weight	~Questions
1. Agentic Architecture & Orchestration	27%	~16
2. Tool Design & MCP Integration	18%	~11
3. Claude Code Configuration & Workflows	20%	~12

Domain	Weight	~Questions
4. Prompt Engineering & Structured Output	20%	~12
5. Context Management & Reliability	15%	~9

The six production scenarios

The question pool is organized around six production scenarios; each sitting draws four. Questions arrive attached to these running systems, so you reason inside a concrete architecture rather than answering isolated definitions. Knowing the six shapes in advance is a large advantage:

1. **Automated Code Review Pipeline.** Agentic loop design with `stop_reason` termination, CI/CD headless mode (`-p` flag), independent review sessions instead of self-review, structured output schemas for review comments, Plan Mode versus direct execution.
2. **Customer Support Escalation System.** Escalation rules (ambiguity escalates, sentiment alone does not), hub-and-spoke orchestration, session management with `--resume`, programmatic human-handoff triggers, why self-reported confidence is unreliable.
3. **Document Extraction at Scale.** Structured output schemas with nullable fields, `"unclear"` enums for ambiguous data, Batch API for overnight bulk work, validation-retry loops, stratified human review sampling.
4. **Multi-Repository Security Audit.** Parallel subagent execution, tool scoping per subagent, error propagation with structured partial results, crash recovery via manifest files, token budgets across parallel loops.
5. **Knowledge Base Q&A System.** Information provenance with claim-source mappings, conflicting-source handling, lost-in-the-middle mitigation, context degradation detection, progressive summarization.
6. **CI/CD Compliance Enforcement.** Hooks in `settings.json`, CLAUDE.md hierarchy for team standards, path-specific rules with glob patterns, headless mode flags.

Part I: Technology

Chapter 1: The Claude API and the agentic loop

1.1 Requests, roles, and statelessness

A Claude API call sends a `messages` array of alternating `user` and `assistant` turns, an optional `system` prompt, and configuration (model, `max_tokens`, `tools`). The single most important fact underneath every exam question: **the API is stateless**. Claude has no memory between calls. The conversation history you send IS the memory. Every request must carry the full history the model needs, including every prior tool result.

1.2 stop_reason controls everything

Each response carries a `stop_reason` that tells your code what Claude intends:

- `"end_turn"`: Claude considers the task complete. Terminate the loop.
- `"tool_use"`: Claude wants a tool executed. Run it, append the result, continue the loop.
- `"max_tokens"`: the response was truncated by the output limit. Handle explicitly; treating truncation as completion silently corrupts results.

The API has a few more values worth knowing: `"refusal"` (Claude declined), `"stop_sequence"` (one of your stop sequences was hit), `"pause_turn"` (a server-side tool loop such as web search paused; send the response back as-is to let Claude continue) and `"model_context_window_exceeded"` (the response filled the context window). Branch on them explicitly instead of treating every unknown value as completion.

The canonical agentic loop:

```
while True:
    response = client.messages.create(
        model="claude-sonnet-4-6",
        messages=messages,
        tools=tools,
    )
    if response.stop_reason == "end_turn":
        break  # task complete
    if response.stop_reason == "tool_use":
        tool_result = execute_tool(response.content)
        messages.append({"role": "assistant", "content": response.content})
        messages.append({"role": "user", "content": tool_result})
```

Anti-pattern (heavily tested): parsing Claude's natural-language output to decide whether to continue ("check if the response contains 'I'm done'"). Natural language parsing is fragile. `stop_reason` is the contract. Similarly, an arbitrary iteration cap (`max_loops = 10`) is a safety net, never the primary termination mechanism; semantic completion via `stop_reason` is.

1.3 Token budgets

Long-running loops need a budget enforced from outside the model:

- Track cumulative input plus output tokens from each response's `usage` field. Note that history grows every iteration, so later calls cost more.
- At ~80% of budget, inject an instruction to wrap up within the next iteration or two (a soft warning).
- At 100%, break the loop programmatically (a hard stop). Never rely on Claude to self-terminate.

```
MAX_TOKENS = 100_000
total = 0
while True:
    response = client.messages.create(...)
    total += response.usage.input_tokens + response.usage.output_tokens
    if total > MAX_TOKENS:
        break  # hard stop, enforced outside the model
    if total > MAX_TOKENS * 0.8:
        messages.append({"role": "user",
                         "content": "TOKEN BUDGET WARNING: summarize findings and conclude."})
```

1.4 Handling truncation and refusal

Two `stop_reason` values break naive response handling, and both appear in exam scenarios. `"max_tokens"` means the response was cut off by the output cap; treating it as a complete answer silently corrupts downstream processing. A refusal means Claude declined the request; code that reads the first content block unconditionally crashes or mishandles it.

```
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=2048,
    messages=messages,
)

match response.stop_reason:
    case "end_turn":
        result = response.content[0].text          # complete answer
    case "max_tokens":
        # Truncated: never parse as complete. Retry with a higher cap,
        # stream, or split the task.
        handle_truncation(response)
    case "tool_use":
        run_tool_loop(response)                    # continue the loop
    case _:
        log_and_escalate(response)                 # refusal and anything new
```

Anti-pattern: `text = response.content[0].text` with no `stop_reason` check. It works in every demo and fails in production the first time an output hits the cap.

1.5 Model selection

Reason by tier, and anchor production answers to the mid tier:

Model	Tier	Pick it when
Claude Haiku	Fastest, cheapest	Simple, high-volume, latency-sensitive tasks (classification, routing)

Model	Tier	Pick it when
Claude Sonnet	Balance of speed and intelligence	The default production workhorse; the safe exam answer for standard workloads
Claude Opus tier	Most capable	The hardest reasoning and long-horizon agentic work where capability beats cost

When a question describes a routine production workload and offers a top-tier model as an answer, cost discipline usually makes the mid-tier model correct. When it describes genuinely hard multi-step reasoning, capability wins.

Chapter 2: Tools and tool_use

2.1 How tool selection actually works

Claude selects tools by reading their **descriptions**. Not the tool name, and not hints buried in the system prompt: the description is the primary routing mechanism. A well-written description contains:

- A one-sentence purpose statement ("Searches a PostgreSQL database for customer records matching the given criteria.")
- Parameter documentation with type, format, and constraints ("customer_id: string, UUID format, required")
- A boundary statement naming what the tool does NOT do and which tool to use instead ("This tool searches existing records. To create records, use create_customer.")
- One or two example invocations
- Error behavior ("Returns { error: 'not_found' } if no records match.")

A complete tool definition puts that description to work alongside a JSON Schema for the inputs:

```
{
  "name": "search_customers",
  "description": "Searches existing customer records in the CRM by name, email, or region.
        Returns up to 20 matches with id, name, email, and account status. Use this to FIND
        existing customers. To create a new customer record, use create_customer instead.
        Returns {\"matches\": []} when no records match; returns {\"error\":
        \"permission_denied\"} if the caller lacks CRM read access.",
  "input_schema": {
    "type": "object",
    "properties": {
      "query":  { "type": "string", "description": "Name, email, or partial match" },
      "region": { "type": "string", "enum": ["NA", "EMEA", "APAC"], "description": "Optional
        region filter" },
      "limit":  { "type": "integer", "minimum": 1, "maximum": 20, "default": 10 }
    },
    "required": ["query"]
  }
}
```

Notice what the description carries: purpose, the boundary against the neighboring tool, and the exact shape of both empty and error results. That last part is what lets the agent distinguish "no customers found" from "not allowed to look", the distinction Section 2.5 tests.

2.2 How many tools

4-5 tools per agent is the reliable range. Selection accuracy degrades significantly as the count grows, and by around 18+ tools Claude begins misrouting calls. When a system needs more capability than five tools can express, the answer is splitting into subagents with scoped tool sets, never loading everything into one agent.

2.3 tool_choice

Value	Behavior	Use when
`"auto"`	Claude may answer in text OR call a tool	Default; let Claude decide
`"any"`	Claude MUST call some tool	You need tool use but the choice can vary
`{"type": "tool", "name": "extract_data"}`	Claude MUST call that specific tool	Forcing structured extraction through a schema

Exam pattern: "How do you ensure Claude always returns structured JSON?" The answer is a schema-enforced mechanism: a tool defined with the JSON schema and forced through `tool_choice`, or structured outputs (the schema in `output_config.format`, or `strict: true` on the tool), which guarantee schema-valid JSON through constrained decoding. A system-prompt instruction to "always respond in JSON" is the trap option.

Current API note: some of the newest models reject forced tool use (`any` or a named `tool`) with a 400 error, and Anthropic points to structured outputs or `auto` plus strict tools instead. The exam's principle stays the same: format guarantees come from a mechanism, never from an instruction.

2.4 Syntax errors versus semantic errors

Schema-enforced output through `tool_use` eliminates **syntax errors**: malformed JSON, missing fields, wrong types. It does nothing about **semantic errors**: wrong values, hallucinated data, misclassifications. Schema validation plus retry handles the first class. Only evaluation and human review catch the second. Questions that blur this line are testing whether you know validation's limits.

2.5 Designing error responses

Tools fail, and how they report failure decides whether an agent can recover:

Error type	isRetryable	Examples
Transient	true	Timeout, service unavailable, rate limit
Validation	false	Invalid input format, missing required field
Business logic	false	Insufficient balance, policy violation
Permission	false	Unauthorized, forbidden resource
Not found	false	Resource does not exist

Rules the exam tests directly:

- **Distinguish access failures from valid empty results.** "Permission denied" and "no results found" demand different agent behavior; collapsing them into an empty list poisons downstream reasoning.
- Return structured error context: failure type, what was attempted, partial results, suggested alternatives.
- Silently returning empty results as success is an anti-pattern because it forecloses recovery.

```
{
  "status": "partial_failure",
  "completed": ["repo_scan", "dependency_check"],
  "failed": {
    "tool": "security_scan",
    "error_type": "timeout",
    "attempted": "Full repository security scan",
    "partial_results": "Scanned 47 of 120 files before timeout",
    "alternatives": ["Retry with reduced scope", "Queue for batch processing"]
  }
}
```

Chapter 3: Multi-agent orchestration

3.1 Hub-and-spoke

The primary multi-agent architecture on this exam. The rules, cold:

- A **coordinator** (hub) manages all communication. Subagents never talk to each other directly.
- Subagents run with **isolated context windows**. They do not inherit the coordinator's history; anything a subagent needs must be passed explicitly in its prompt.
- The coordinator can emit multiple subagent invocations in one response for **parallel** execution, or wait on dependencies for **sequential** execution.
- Failure is **isolated**: one subagent crashing does not take down the coordinator or its siblings.

Aspect	Hub-and-spoke	Flat (all-in-one)
Context	Each subagent gets only what it needs	One agent holds everything
Failure blast radius	One subagent	The whole system
Tools per agent	4-5 scoped	Everything loaded, selection degrades
Scaling	Add subagents independently	Refactor the monolith
Fits	Complex multi-domain work	Simple single-domain tasks

3.2 Subagent design principles

- **Single responsibility** per subagent (security scanning, dependency analysis, code quality).
- **Scoped tools**: only the 4-5 tools that subagent's job requires.
- **Explicit contracts**: define what the coordinator passes in and what comes back.
- **Timeouts** per subagent; the coordinator never waits indefinitely.
- **Result normalization**: subagents return one standardized format so the coordinator merges without per-agent parsing.

3.3 Parallel, sequential, hybrid

Pattern	Use when	Example
Sequential	Step N's output feeds step N+1	Research → Analyze → Recommend
Parallel	Steps are independent	Scan five repositories simultaneously
Hybrid	Mixed dependencies	Parallel scans → sequential merge → final report

3.4 Task decomposition

Two families:

- **Fixed pipelines**: a predetermined sequence of stages. Predictable, testable, right when the workflow is known in advance.
- **Dynamic decomposition**: the coordinator plans subtasks at runtime based on what it discovers. Right for open-ended investigations; costs more tokens and needs tighter budgets.

Over-decomposition is a tested trap: splitting trivially small work across subagents adds orchestration overhead without any isolation benefit.

3.5 State machines and session management

Complex workflows model cleanly as state machines: states like planning, executing, waiting_for_tool, reviewing, escalating, complete, failed; transitions driven by `stop_reason` values, tool results, and validation outcomes; guards as programmatic preconditions. The payoff is crash recovery: persist the state, resume from it.

Session mechanics that appear in questions:

- `--resume <session>` continues a prior session with full history intact.
- `fork_session` branches an independent copy from a shared baseline; changes in the fork never propagate back.

3.6 Orchestration in code

The coordinator patterns above, as they actually look. Parallel fan-out means emitting several delegations in one turn and merging when all return; sequential means awaiting each dependency:

```
# Parallel: independent subtasks, spawned together, merged once
scan_prompts = [scanner_prompt(repo, findings_schema) for repo in repos]
results = run_subagents_parallel(scan_prompts, timeout_s=300)   # isolated contexts
merged = merge_findings([r for r in results if r is not None])  # tolerate failures

# Sequential: B needs A's output, so A completes first
outline = run_subagent(outline_prompt(document))
draft   = run_subagent(draft_prompt(document, outline))         # outline passed explicitly
```

The two rules the exam checks are visible right in the shape of the code: every subagent receives its inputs **explicitly in its prompt** (nothing is inherited), and a failed subagent becomes a `None` to tolerate rather than an exception that destroys its siblings' work.

A structured handoff, whether subagent-to-coordinator or agent-to-human, carries state rather than prose:

```
{
  "handoff_reason": "policy_ambiguity",
  "conversation_summary": "Customer requests refund of $840 for order 19384, outside the 30-day
      window by 3 days",
  "attempted": ["standard_refund_policy_check", "manager_exception_lookup"],
  "blocking_question": "Does the loyalty-tier exception apply to window overruns under 7
      days?",
  "customer_sentiment": "calm",
  "urgency": "normal"
}
```

Chapter 4: Model Context Protocol (MCP)

4.1 What MCP is

MCP is the open protocol standardizing how models discover and call external tools and data sources. An MCP server exposes capabilities; any MCP-capable client (Claude Code, the API via connectors, other vendors' agents) can consume them without custom integration.

4.2 The three capability types

The exam expects you to keep these apart:

- **Tools**: functions Claude can call, with parameters and results (`create_issue`, `search_documents`).
- **Resources**: read-only content catalogs Claude can browse. Data, never actions (GitHub issues, documentation, database schemas).
- **Prompts**: reusable prompt templates the server provides, encapsulating domain-specific prompt engineering.

The three primitives in server code, using the Python SDK's FastMCP style:

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("support-kb")

@mcp.tool()
def search_tickets(query: str, status: str = "open") -> str:
    """Search support tickets by keyword. Use status='closed' for resolved tickets."""
    return json.dumps(ticket_store.search(query, status))

@mcp.resource("kb://articles/{article_id}")
def get_article(article_id: str) -> str:
    """A knowledge-base article, addressable by id, browsable by the client."""
    return kb.fetch(article_id)

@mcp.prompt()
def triage_ticket(ticket_text: str) -> str:
    """Reusable triage prompt with the team's severity rubric baked in."""
    return f"Classify severity using the P1-P4 rubric...\n\nTicket:\n{ticket_text}"

mcp.run()
```

One server, three capability types: the tool is an **action** the model invokes, the resource is **data** the client can browse and attach, and the prompt is packaged **know-how**. Exam questions describe a requirement and ask which primitive fits; the answer follows from whether the requirement is an action, data, or a reusable instruction set.

4.3 Configuration and scope

File	Scope	Version controlled	Use
`.mcp.json`	Project	Yes	Team-wide tool configuration
`~/.claude.json`	User	No	Personal preferences, experiments

```json
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    }
  }
}
```

Two rules with exam weight: secrets go in via `${ENV_VAR}` expansion and are never committed raw, and project-level config is the answer whenever a whole team needs the same servers.

Chapter 5: Claude Code

The most heavily configured surface on the exam (20% as its own domain, and it reappears inside scenarios).

5.1 The CLAUDE.md hierarchy

Location	Scope	Version controlled	Loaded
`~/.claude/CLAUDE.md`	User (personal)	No	Always, for this user only
`PROJECT_ROOT/CLAUDE.md` or `.claude/CLAUDE.md`	Project (team)	Yes	Always, for everyone
`subdirectory/CLAUDE.md`	Directory	Yes	Only when working in that directory

The trap the exam loves: a developer puts team standards in `~/.claude/CLAUDE.md` and wonders why teammates ignore them. User-level config is personal and never ships with the repo. Team instructions belong in the project-level file, committed to version control.

Content guidance: coding standards, test commands, architectural decisions, review criteria. Keep rules specific and actionable ("Run `npm test` before committing"), put the most important ones at the top, and never put secrets in it.

5.2 Path-specific rules

`.claude/rules/` holds rule files with a `paths:` glob list in frontmatter; each loads only when Claude edits matching files:

```
---
paths:
  - "src/api/**/*.ts"
---
All API endpoints must validate input using zod schemas.
```

5.3 Skills and slash commands

Skills are reusable prompt-based extensions with YAML frontmatter. The fields that get tested:

```
---
name: "review-security"
description: "Performs a security-focused code review"
context: fork          # runs in an isolated subagent
allowed-tools: [Read, Grep, Glob]
argument-hint: "file path or directory to review"
---
```

`context: fork` isolates the skill in a subagent so verbose output does not pollute the main conversation. `allowed-tools` applies least privilege.

5.4 Hooks

Hooks run shell commands at lifecycle events. They are the exam's canonical example of programmatic enforcement. The core events:

Event	Fires	Typical use
PreToolUse	Before a tool executes	Block dangerous operations, gate compliance requirements
PostToolUse	After a tool completes	Linters, formatters, output validation
UserPromptSubmit	When the user submits a prompt	Inject context, screen inputs
Stop	When Claude is about to finish	Verify required steps actually ran
PreCompact	Before history compaction	Preserve critical state
SessionStart / SessionEnd	Session lifecycle	Setup, logging
SubagentStop	A subagent finishes	Validate subagent output

Configuration in `settings.json` nests event → matcher group → hook handlers. The matcher filters by tool name (exact, `|`-separated, or regex), and an `if` field narrows to specific invocations:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "if": "Bash(rm *)",
            "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/block-rm.sh"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [ { "type": "command", "command": "npx eslint --fix" } ]
      }
    ]
  }
}
```

A hook script controls the outcome through its exit code and output: **exit 0** allows (and can return structured JSON for fine-grained decisions), **exit 2 blocks the action**, with the stderr text or a JSON `permissionDecision` explaining why:

```bash
#!/bin/bash
# .claude/hooks/block-rm.sh: deny recursive deletes, allow everything else
COMMAND=$(jq -r '.tool_input.command')
if echo "$COMMAND" | grep -q 'rm -rf'; then
  jq -n '{ hookSpecificOutput: {
    hookEventName: "PreToolUse",
    permissionDecision: "deny",
    permissionDecisionReason: "Destructive command blocked by policy"
  } }'
else
  exit 0
fi
```

Two details worth having cold: there is no dedicated commit event, so "always run tests before committing" is a `PreToolUse` hook matching `Bash` with an `if` on `git commit`, and hooks defined in the committed project `settings.json` reach the whole team while `~/.claude/settings.json` hooks are personal.

Whenever a question asks how to **guarantee** a behavior (tests always run before commit, `rm -rf` never executes), the answer is a hook. A CLAUDE.md instruction is guidance the model probably follows; a hook is a mechanism it cannot bypass.

5.5 Imports, local files, and session commands

Four mechanics round out the configuration surface, and each has appeared in scenario questions:

- **`@path` imports.** A CLAUDE.md can pull in other files with `@path/to/file` syntax: `See @README for the project overview and @docs/git-instructions.md for the git workflow.` Imports resolve relative to the importing file, recurse up to four hops deep, and are skipped inside backticks and code blocks. Imported files load at launch, so imports organize content without saving context.
- **`CLAUDE.local.md`.** Personal, project-specific instructions that load alongside the project CLAUDE.md and belong in `.gitignore`: sandbox URLs, preferred test data, anything true for you and wrong for the team. This completes the placement question: team-wide → committed project file; personal-everywhere → `~/.claude/CLAUDE.md`; personal-this-project → `CLAUDE.local.md`.
- **`/compact` and what survives it.** Compaction summarizes conversation history to reclaim context. The project-root CLAUDE.md survives: it is re-read from disk and re-injected afterward. Instructions given only in conversation do not survive, which is exactly why durable rules belong in files rather than chat.
- **`/init`, `/memory`, `/context`.** `/init` generates a starting CLAUDE.md by analyzing the codebase, `/memory` opens the memory files for editing, and `/context` shows which files actually loaded into the current session, the first debugging step when Claude ignores an instruction.

5.6 Plan Mode versus direct execution

Plan Mode	Direct execution
Multi-file coordinated changes	Single-file fixes
Multiple valid architectural approaches	A clear stack trace pointing at the bug
Unfamiliar codebase exploration	Simple additions to known files

Plan Mode	Direct execution
Cross-cutting refactors	Typos and small edits

5.7 Built-in tools

Tool	Purpose	Detail with exam weight
Grep	Search file contents	Regex, ripgrep-based
Glob	Match file paths	Patterns like `**/*.ts`
Read	Read files	Line ranges, images, PDFs
Edit	Targeted modification	Needs a unique text anchor to match
Write	Create or overwrite	Edit for changes, Write for new files
Bash	Shell commands	Working directory resets between calls

Investigate incrementally: Grep/Glob to locate, Read the relevant region, then Edit. Reading entire large files into context when a search would do is the anti-pattern being probed.

5.8 CI/CD and headless mode

- `-p` / `--print`: non-interactive mode; without it the pipeline hangs waiting for input. The essential flag.
- `--output-format json`: machine-parseable output.
- `--tools "Read,Grep,Glob"`: limit the run to those built-in tools (least privilege in automation).
- `--allowedTools` is a different flag: it pre-approves tools so they run without a permission prompt, and `--disallowedTools` removes or denies specific tools. A CI job that needs both restriction and no prompts uses `--tools` for the first and `--allowedTools` for the second.

```
claude -p "Review this PR for security issues" --output-format json --tools "Read,Grep,Glob"
```

Two principles ride along: include prior findings in the prompt so repeated reviews report only new issues, and separate generation from review. A model reviewing code it just wrote, in the same session, retains the reasoning that produced it and is biased toward approving it. Independent review sessions fix this.

Chapter 6: Prompt engineering and structured output

6.1 Explicit criteria beat vague guidance

"Be conservative" does not reduce false positives, and "be thorough" does not improve coverage. Working instructions name the criteria: "Only flag issues with severity high or above that have a direct security impact." "Check for SQL injection, XSS, CSRF, and authentication bypass." The test for a good criterion: someone reading the output can verify whether it was met.

System prompt shape: role first, then numbered explicit criteria, then boundary statements ("Do NOT suggest style changes"), then the exact output format.

6.2 Few-shot examples: what they can and cannot do

Use 2-4 targeted examples that demonstrate classification reasoning and output format, including at least one example of what NOT to flag and one tricky edge case.

What few-shot cannot do, and the exam checks: **enforce ordering or compliance**. Examples showing tools called in the correct order do not guarantee that order. Ordering is a compliance requirement, and compliance requirements get programmatic enforcement (hooks, gates, orchestration).

6.3 Prompt chaining

Complex work splits into focused chained prompts, each doing one job: extract → validate → enrich → format. Each stage gets its own instructions and its own validation. One giant prompt doing everything at once is less reliable and much harder to debug.

6.4 The interview pattern and self-correction

Two named patterns extend chaining and both show up in scenario questions:

- **The interview pattern** inverts the flow: instead of guessing what the model needs, you instruct it to ask. "Before drafting the migration plan, ask me up to five questions about constraints you need and cannot infer from the repository." This fits tasks where requirements live in a human's head, and it beats a long speculative prompt that guesses wrong.
- **Self-correction as a separate pass.** After generation, a follow-up prompt asks the model to check its own output against explicit criteria: "Review the extraction above against the schema. List any field where the value is not directly supported by the source text, then output the corrected JSON." A self-correction pass with named criteria catches real errors; the same instruction folded into the original prompt ("double-check your work") mostly does not. Note the boundary from Section 6.7: self-correction against *criteria* helps, while self-*review* of open-ended quality in the same context stays biased, which is why independent review instances exist.

6.5 Schema design for structured output

Force extraction through a schema (structured outputs, or a tool defined with the schema and forced with `tool_choice` on models that allow forcing), and design the schema for the real world:

- **Nullable fields** for data that may not exist in the source.
- **`"unclear"` enum values** for genuinely ambiguous fields.
- **`"other"` plus a detail string** for categories outside the predefined enum.

Every field should have a graceful "I don't know" path. A required, non-nullable field for information that is sometimes absent guarantees validation failures no retry can fix.

6.6 Validation and retry

When extraction fails validation, the retry prompt carries three things: the original document, the failed output, and the specific validation errors, with an instruction to fix only those errors.

The boundary that gets tested: **retries fix execution errors, never information gaps.** If Claude misformatted a date, a retry fixes it. If the source document simply lacks the revenue figure, no number of retries will produce it; the nullable schema absorbs it instead.

6.7 Batch API

Aspect	Batch API	Real-time API
Cost	50% cheaper	Standard
Processing	Up to 24 hours	Seconds
Latency SLA	None	Yes
Multi-turn	No, single request	Yes
Correlation	`custom_id` per request	Conversation/session
Fits	Overnight reports, weekly audits, bulk extraction	User-facing, interactive

The all-time trap: proposing Batch for anything a user is waiting on. No SLA means possibly 24 hours. The reverse trap also appears: paying real-time prices for overnight bulk work that Batch would halve.

6.8 Multi-instance review

Self-review in the same context is biased: the model retains the reasoning that produced the output and tends to confirm it. Production review uses independent instances that see only the artifact, not the conversation that generated it. For code, split per-file passes (local issues) from a cross-file integration pass (systemic issues).

Chapter 7: Context management in production

7.1 Lost-in-the-middle

Models attend well to the beginning and end of the context window and poorly to the middle. Consequences the exam builds questions on:

- Critical information belongs at the **beginning** (or end), never buried mid-context.
- "Increase the context window" is not a fix for attention problems and is a marked trap answer. Restructure instead: extract key facts into a block at the top, or split into focused passes.

7.2 Keeping context lean

- **Trim tool outputs** to the fields the next step needs before appending them to history.
- **Prune ceremony**: "file saved successfully" messages and other irrelevant successes.
- **Progressive summarization**: every N tool calls, replace detailed history with a summary.
- **Key-fact extraction**: pull names, numbers, dates, and decisions into a persistent structured block at the prompt's top.
- **Scratchpad files**: persist findings outside the window entirely, and read back only what is needed.
- **Subagent offloading**: ten files to analyze means ten focused subagent contexts returning summaries, never one context holding all ten.

7.3 Position-aware input and scratchpads

Position-aware input applies the attention curve deliberately. The same facts, restructured so the critical block leads:

```
prompt = f"""CASE FACTS (authoritative, extracted from prior steps):
- Customer: {facts['customer_id']}, tier: {facts['tier']}
- Order {facts['order_id']}: ${facts['amount']}, delivered {facts['delivered']}
- Policy window: 30 days; request is {facts['days_over']} days past it

TASK: {task}

SUPPORTING MATERIAL (reference only):
{long_tool_outputs}"""
```

A scratchpad moves state out of the window entirely. The agent appends findings to a file as it works and reads back only what the next step needs:

```
def note(finding: dict):
    with open("scratchpad.jsonl", "a") as f:
        f.write(json.dumps(finding) + "\n")

def recall(topic: str) -> list[dict]:
    return [json.loads(l) for l in open("scratchpad.jsonl")
            if topic in l]          # selective read-back, never the whole file
```

The pattern's point is the selective read-back: a scratchpad that gets dumped wholesale into every prompt has just reinvented the bloated context it was meant to prevent.

7.4 Detecting context degradation

Signals that the model has lost the thread:

- Inconsistent answers to the same question within one session
- References to "typical patterns" instead of specific findings from this conversation
- Re-running tools whose results are already in history
- Citing information never provided

Remediation: scratchpads, subagent offloading, and compaction of the conversation while preserving essentials.

Chapter 8: Reliability, escalation, and recovery

8.1 Escalation and human-in-the-loop

Escalate when:

- The customer explicitly asks for a human
- Policy is **ambiguous** and no clear rule applies
- The agent has made no progress after multiple attempts (deterministic loop detection)
- The action is irreversible (deletions, refunds above threshold)

- Sources conflict in a way that cannot be resolved programmatically

Do NOT escalate merely because:

- The customer sounds frustrated (sentiment is not a trigger)
- The model's self-reported confidence is low (**confidence scores are poorly calibrated**, and the exam tests this trap by name)
- The task is complex but sits inside clear policy

The mental model: **complexity is the agent's job; ambiguity is a human's.** Handoffs are structured: state, history summary, what was attempted, why escalation fired.

8.2 Error propagation in chains

Covered mechanically in Chapter 2.5; the orchestration-level rules: preserve partial results (three of four successful scans are still valuable), give the coordinator structured context to make recovery decisions, and never let one failure cascade into unrelated subagents.

8.3 Crash recovery

Long-running agents checkpoint to a **manifest file**: completed steps, pending steps, intermediate results, timestamp. On restart, the coordinator loads the manifest into the new prompt and resumes from the checkpoint. Two supporting properties:

- **Idempotency**: tool calls safe to replay without duplicate side effects, so re-executing from a checkpoint is harmless.
- **Circuit breaker**: after N consecutive failures of a tool, stop retrying, switch to a fallback, reset after cooldown.
- **Graceful degradation**: a non-critical tool failing should narrow the output ("results exclude the security scan, which was unavailable"), never abort the workflow.

8.4 Coverage annotations

When an agent reports on work spanning many items (files reviewed, repos scanned, documents processed), require an explicit coverage annotation rather than trusting silence:

```
{
  "reviewed": ["auth.py", "sessions.py", "tokens.py"],
  "skipped": [
    { "file": "legacy_crypto.py", "reason": "exceeds single-pass size; queued for focused pass"
        }
  ],
  "coverage": "3 of 4 files"
}
```

Without the annotation, "no issues found in the audit" and "no issues found in the 60% of the audit that actually ran" are indistinguishable, which is the failure the exam probes. The same principle produced the structured partial-failure responses in Chapter 2.5; coverage annotations extend it from errors to normal operation.

8.5 Provenance

When synthesizing from multiple sources: map every claim to its source (URL, document, excerpt, date). When sources conflict, annotate BOTH values with attribution and surface the conflict; silently picking one is the tested anti-pattern. Distinguish quotes, paraphrases, and inferences, and track recency.

8.6 Quality assurance beyond the average

A 97% aggregate accuracy can conceal complete failure on a rare document type: if 90% of inputs are easy, the hard 10% can be failing entirely behind a comfortable average. The tested answer is **stratified sampling**: sample by document type AND by field, set thresholds per stratum, and route low-confidence extractions to human review rather than accepting them silently.

Part II: Exam domain notes

Revision-oriented summaries. Each domain lists what you must know (facts) and what you must be able to do (judgment), then the domain's anti-patterns.

Domain 1: Agentic Architecture & Orchestration (27%)

Know: the agentic loop and `stop_reason` semantics; hub-and-spoke rules (isolated contexts, explicit data passing, coordinator-mediated communication); parallel vs sequential vs hybrid execution; fixed vs dynamic decomposition; state-machine modeling; `--resume` vs `fork_session`; token budget mechanics (soft warning ~80%, programmatic hard stop).

Be able to: design a loop that terminates on semantics rather than iteration caps; decide what context a subagent genuinely needs; choose parallel or sequential from data dependencies; recognize over-decomposition; place a token budget outside the model's control.

Anti-patterns: parsing prose for termination; iteration caps as primary stop; sharing full coordinator context with subagents; few-shot examples to enforce tool ordering; letting one subagent failure cascade.

Domain 2: Tool Design & MCP Integration (18%)

Know: descriptions are the routing mechanism; the 4-5 tool sweet spot and degradation at ~18+; the three `tool_choice` values; the isRetryable error taxonomy; `.mcp.json` (project, committed) vs `~/.claude.json` (user, personal); `${ENV_VAR}` secret expansion; MCP tools vs resources vs prompts; built-in tool purposes.

Be able to: write a description with purpose, parameters, boundaries, examples, and error behavior; classify an error's retryability; design a structured error response with partial results; distinguish access failure from empty result; choose the correct MCP capability type for a requirement.

Anti-patterns: routing by tool name; the everything-agent with 18+ tools; raw secrets in committed config; generic "error occurred" responses; empty-result-as-success.

Domain 3: Claude Code Configuration & Workflows (20%)

Know: the CLAUDE.md hierarchy and which levels are version controlled; the team-standards-in-user-config trap; `.claude/rules/` path scoping; skill frontmatter (`context: fork`, `allowed-tools`, `argument-hint`); hook events and their `settings.json` syntax; Plan Mode criteria; headless flags (`-p`, `--output-format json`, `--tools` to restrict, `--allowedTools` to pre-approve).

Be able to: place a rule at the correct hierarchy level; convert a "Claude must always/never" requirement into the right hook; scope a skill's tools to least privilege; decide Plan Mode vs direct execution from the change's shape; wire a CI review that stays non-interactive and reports only new findings.

Anti-patterns: team standards in `~/.claude/CLAUDE.md`; CLAUDE.md guidance where a hook is required; skills with unrestricted tools; CI invocations without `-p`; self-review in the generation session.

Domain 4: Prompt Engineering & Structured Output (20%)

Know: explicit-testable-criteria doctrine; few-shot's scope (format and reasoning yes, compliance no); prompt chaining stages; nullable/unclear/other schema design; the validation-retry loop's inputs; execution errors vs information gaps; the full Batch vs real-time table; multi-instance review rationale.

Be able to: rewrite vague guidance into testable criteria; build a 2-4 example few-shot set including negative examples; design a schema that survives missing and ambiguous data; construct a retry prompt; route a workload correctly between Batch and real-time; set up unbiased review.

Anti-patterns: "be conservative/thorough" as instructions; few-shot for ordering; retrying to conjure absent data; Batch behind a waiting user; prompt-only JSON formatting; enums with no escape value.

Domain 5: Context Management & Reliability (15%)

Know: lost-in-the-middle and its mitigations; trimming, pruning, progressive summarization, key-fact blocks, scratchpads, subagent offloading; degradation signals; escalation triggers and non-triggers; manifest-based crash recovery; idempotency, circuit breakers, graceful degradation; claim-source provenance and conflict annotation; stratified vs random sampling.

Be able to: restructure a prompt so critical facts lead; detect degradation from behavioral signals; write an escalation policy that fires on ambiguity and irreversibility, never on sentiment or self-confidence; design a checkpoint manifest; expose a per-stratum quality view that an aggregate metric hides.

Anti-patterns: "bigger context window" as an attention fix; no state persistence in long workflows; aggregate accuracy without stratification; discarding partial results; silent conflict resolution between sources.

Part III: Exam preparation

The preparation ladder

Five layers, in order. Each builds on the previous, and the free layers take you surprisingly far. Anthropic retired its official practice exam when the exams moved to Pearson VUE; the sample questions in the official exam guide are what remains.

Step	Resource	Cost
1. Learn the vocabulary	Anthropic Partner Academy courses, the official exam guide, and the Anthropic Academy courses mapped in Resources	Free
2. Go to the source	docs.anthropic.com: agents, tool use, MCP, Claude Code	Free
3. First calibration	30 practice questions with explanations in this repo, also as a timed practice test	Free
4. Full calibration	Six full-length scenario-format practice tests and a 500-card flashcard deck on Preporato, built on the exact domain weights; the page has free sample questions to try first	Paid
5. Prove it hands-on	11 auto-graded build-and-submit projects: agentic loops, hub-and-spoke systems, MCP servers, extraction pipelines	Paid

Steps 1-3 cost nothing and tell you honestly where you stand. Steps 4-5 are how you close the gap they reveal.

The Golden Rule and five mental models

The Golden Rule: programmatic enforcement beats prompt-based guidance. On any question where both appear, the mechanism (hook, gate, schema validation, retry loop, hard stop) beats the instruction. Prompts are probabilistic; mechanisms are deterministic. If a behavior must ALWAYS happen, the answer is never "add it to the system prompt."

Five mental models for evaluating answer choices:

1. **The Determinism Test.** Can this solution fail silently? Then it needs programmatic enforcement.
2. **The Isolation Principle.** Does this subagent need all this context? Minimum necessary context is almost always the right answer.
3. **The Recovery Question.** If this fails at 3 AM with nobody watching, can the system recover? Prefer answers with structured errors, partial results, and checkpoints.
4. **The Attention Budget.** Is the critical information buried mid-context? "Process everything in one pass" loses to focused passes with an integration step.
5. **The Calibration Check.** Does this rely on Claude assessing its own confidence or quality? Replace with external validation and deterministic thresholds.

The seven anti-patterns

Anti-pattern	Why wrong	Correct approach
Few-shot examples for tool ordering	Ordering is compliance; few-shot is probabilistic	Hooks, gates, orchestration
Self-reported confidence for escalation	Poorly calibrated	Deterministic thresholds and rule triggers
Batch API for user-facing blocking flows	No SLA, up to 24h	Real-time API with streaming
Bigger context = better attention	Lost-in-the-middle persists	Focused passes, key facts up front
Silent empty results on failure	Forecloses recovery	Structured error context with partials
All tools on one agent	Selection degrades from ~18 tools	4-5 scoped tools, split into subagents
Prompt-only JSON formatting	Probabilistic, malformed output possible	Schema-enforced output (structured outputs or a schema-defined tool) plus validation of the values

Critical distinctions

A	B	The difference
`stop_reason: end_turn`	`stop_reason: tool_use`	Done vs continue the loop
`tool_choice: any`	`tool_choice: auto`	Must call some tool vs may answer in text
`~/.claude/CLAUDE.md`	`.claude/CLAUDE.md`	Personal vs team (version controlled)
`context: fork`	Default skill context	Isolated subagent vs main conversation
Syntax errors	Semantic errors	Schemas fix the first; review catches the second
`isRetryable: true`	`isRetryable: false`	Transient vs validation/permission/logic
Batch API	Real-time API	50% cheaper, up to 24h, no SLA vs instant
Plan Mode	Direct execution	Multi-file/architectural vs single-file/obvious
Hooks	CLAUDE.md instructions	Deterministic vs probabilistic
Stratified sampling	Random sampling	Catches per-type failures vs masks them
`--resume`	`fork_session`	Continue same session vs independent branch
Access failure	Valid empty result	Different failures demanding different recovery
Token hard stop	Token soft warning	Programmatic break vs wrap-up instruction

Exam-day strategy

Time. 120 minutes, 60 questions, so 2 minutes average, and scenario sections run long. First ~10 minutes: read all six scenario introductions and choose your four strongest. Minutes 10-90: the four scenarios, capped at ~20 minutes each. Minutes 90-110: standalone questions. Final 10: review flags; change answers only on certain error.

Per question, identify three things before reading the choices: which domain is being tested, whether it asks for the right approach or the anti-pattern, and whether the Golden Rule applies.

Named trap patterns. "Update the system prompt to always..." when a mechanism exists. "Increase the context window" for attention problems. "Use Claude's confidence score" for escalation. "Add more few-shot examples" for ordering compliance. "97% accuracy proves it works" without stratification.

Worked questions

Twelve fresh questions in the exam's style, with reasoning. (These are written for this guide; timed full-length practice is linked in Resources.)

Q1. Your agentic loop sometimes runs forever on ambiguous tasks. A teammate proposes `max_iterations = 15` as the fix. What is the strongest objection?

- A) 15 is too low for complex tasks
- B) Iteration caps address the symptom; termination should be driven by `stop_reason` semantics, with the cap kept only as a safety net
- C) The loop should instead check whether Claude's text says the task is finished
- D) Token budgets make iteration caps redundant

Answer: B. The loop's primary contract is `stop_reason`. A cap is a legitimate backstop but a poor primary mechanism (A argues about the number, missing the principle; C is the prose-parsing anti-pattern; D confuses two complementary safety nets).

Q2. A support agent must hand off to a human whenever a refund exceeds $500. Where does this rule live?

- A) In the system prompt: "Always escalate refunds above $500"
- B) In a few-shot example showing a $600 refund being escalated
- C) In a programmatic gate that checks the refund amount before the refund tool executes
- D) In the tool description for the refund tool

Answer: C. "Whenever" means compliance, and compliance is enforced by mechanism (a check in code before the refund tool runs, the same idea as a `PreToolUse` hook). A and B are probabilistic; D improves selection but cannot enforce a threshold.

Q3. Claude must return valid JSON matching a strict schema for every extraction. Which combination guarantees the format?

- A) A system prompt with the schema pasted in and an instruction to follow it exactly

- B) Schema-enforced output (structured outputs, or an extraction tool defined with the JSON schema and forced on models that allow it), plus validation of the values
- C) Few-shot examples of correctly formatted output
- D) Lowering temperature to 0

Answer: B. Schema enforcement eliminates syntax errors, and validation catches what a schema can't, such as wrong values. A and C are probabilistic; D reduces variance without guaranteeing structure.

Q4. A document-extraction pipeline keeps failing validation on the `revenue` field for a minority of documents. Investigation shows those documents genuinely contain no revenue figure. The fix?

- A) Retry failed documents up to five times
- B) Add few-shot examples with revenue correctly extracted
- C) Make `revenue` nullable in the schema and let downstream logic handle its absence
- D) Route those documents to a more capable model

Answer: C. This is an information gap. Retries (A) and examples (B) cannot create data absent from the source, and a stronger model (D) cannot either. The schema must absorb reality.

Q5. Your coordinator spawns four repository scanners in parallel. One times out. Which return value from the failed scanner best serves the system?

- A) An empty findings list, so the merge step is not disrupted
- B) A structured error naming the timeout, what was attempted, the 47 of 120 files it did scan, and retry alternatives
- C) A raised exception that halts the audit for investigation
- D) "Scan failed" with a null result

Answer: B. Partial results plus context enable an informed recovery decision. A is empty-as-success, D is context-free failure, and C throws away three successful scans.

Q6. (Select TWO) Which placements make a team-wide "always run tests before commit" requirement actually binding for everyone?

- A) `~/.claude/CLAUDE.md`
- B) The project's committed `.claude/CLAUDE.md`
- C) A `PreToolUse` hook in the project's committed settings that runs the tests whenever Claude runs `git commit`
- D) A note in the repository README
- E) A few-shot example in each developer's personal config

Answer: B and C. Project-level, version-controlled placement reaches the team (B), and the hook makes it enforcement rather than guidance (C). C is the stronger of the two; B alone is probabilistic but at least correctly scoped. A and E are personal-scope, D is invisible to Claude in most workflows.

Q7. A nightly compliance job summarizes 40,000 support transcripts. Cost is the concern; nobody reads results before morning. Which delivery mechanism?

- A) Real-time API with prompt caching

- B) Real-time API with streaming
- C) Batch API with `custom_id` correlating each transcript
- D) An agentic loop processing transcripts sequentially

Answer: C. Overnight, bulk, no one waiting: Batch's 50% saving with no SLA fits exactly, and `custom_id` maps results back. A and B pay real-time prices for latency nobody uses; D adds orchestration where none is needed.

Q8. After 30 minutes of a long code investigation, Claude begins re-running searches it already ran and describing "common patterns in codebases like this" instead of citing files it read. What is happening, and what is the right response?

- A) The model is hallucinating; switch to a larger model
- B) Context degradation; persist key findings to a scratchpad, compact the history, and offload remaining subtasks to focused subagents
- C) The context window is too small; raise it
- D) Tool misconfiguration; re-register the search tools

Answer: B. Repeated tool calls and generic-pattern language are the named degradation signals. C is the bigger-context trap; A and D misread the signals.

Q9. Your extraction system reports 97% accuracy overall, but a customer complains that purchase orders in one regional format are consistently wrong. What does the QA process need?

- A) A higher overall accuracy target, such as 99%
- B) Stratified sampling by document type and field, with per-stratum thresholds
- C) A second model voting on every extraction
- D) Larger random samples

Answer: B. The aggregate hides a per-type failure; only stratification exposes it. A and D keep measuring the same masked average; C adds cost without changing what is measured.

Q10. A reviewer agent should evaluate code that a generator agent just produced. Which setup yields the most trustworthy review?

- A) Ask the generator to review its own output in the same session, since it has full context
- B) A fresh, independent session that receives only the code, running per-file passes plus one cross-file integration pass
- C) The generator reviews in the same session but with a system prompt instructing it to be objective
- D) An independent session that receives the code plus the generator's full reasoning transcript

Answer: B. Independence requires not inheriting the generation reasoning; per-file plus integration passes catch both local and systemic issues. A and C keep the bias, and D reintroduces it through the transcript.

Q11. A compliance team requires that Claude Code can never execute a shell command containing `curl` against production hosts, for any developer, with no exceptions. Which implementation meets the requirement?

- A) A rule in the committed project CLAUDE.md: "Never run curl against production hosts"
- B) A `PreToolUse` hook in the committed project settings, matching the Bash tool, whose script inspects the command and exits 2 (or returns a deny decision) on violations
- C) A `PostToolUse` hook that reports violations to the security channel
- D) A note in each developer's `~/.claude/CLAUDE.md`

Answer: B. "Never, for any developer, no exceptions" is enforcement, and only a blocking `PreToolUse` hook in *committed project* settings delivers it: deterministic, team-wide, and evaluated before execution. A is probabilistic guidance, C fires after the damage, and D is personal scope that reaches nobody else.

Q12. A developer keeps personal sandbox URLs and test credentials paths in the project's committed CLAUDE.md "so Claude remembers them", and teammates complain about the noise. Where does that content belong?

- A) A subdirectory CLAUDE.md
- B) The system prompt of each session
- C) `CLAUDE.local.md` at the project root, listed in `.gitignore`
- D) An `@import` from the committed CLAUDE.md to the developer's home directory

Answer: C. Personal, project-specific content has a dedicated home: `CLAUDE.local.md`, loaded alongside the project file and kept out of version control. A still ships to the team, B does not persist, and D commits a reference to one person's home directory into the shared file, which breaks for everyone else.

30-day study plan

- **Days 1-7: official baseline.** Check that your employer is in the Claude Partner Network (registration needs your company email), complete the Partner Academy architect courses, read the official exam guide, and map your experience against the five domains.
- **Days 8-21: build.** Work through docs.anthropic.com with a project open: an agent with tools and a real loop, an MCP server, a Claude Code setup with CLAUDE.md hierarchy, one hook, one skill, and a headless CI invocation. Part I of this guide is your reading companion; the exam rewards people who have hit these failure modes personally. If you want these builds checked, the hands-on projects (paid) cover the same ground with automatic rubric grading.
- **Days 22-30: calibrate.** Start with the 30 practice questions in this repo to see the format, then take full-length, timed, scenario-format practice tests. Review per-domain results, re-read the matching Part II notes, and spend the final days on your two weakest domains. The night before, review Part III's tables and the cheat sheet.

The expanded week-by-week version: 30-day CCA-F study plan.

Pre-exam checklist

You are ready when every line is true:

- [ ] I can explain the agentic loop and why `stop_reason` (never prose parsing or iteration caps) controls termination
- [ ] I know the hub-and-spoke rules: isolated contexts, explicit data passing, coordinator-mediated communication, isolated failure
- [ ] I can choose parallel vs sequential vs hybrid execution from data dependencies
- [ ] I know escalation triggers (ambiguity, irreversibility, explicit request, no progress) and non-triggers (sentiment, self-reported confidence)
- [ ] I know tool descriptions route selection, and 4-5 scoped tools is the reliable range
- [ ] I can state all three `tool_choice` values and their use cases
- [ ] I can classify errors by retryability and design a structured partial-failure response
- [ ] I know the CLAUDE.md hierarchy, what is version controlled, and the team-config trap
- [ ] I know `.mcp.json` vs `~/.claude.json` and `${ENV_VAR}` secret expansion
- [ ] I can distinguish MCP tools, resources, and prompts
- [ ] I know hook events and why hooks beat CLAUDE.md for guarantees
- [ ] I know `context: fork`, `allowed-tools`, and path-scoped rules
- [ ] I know the headless flags: `-p`, `--output-format json`, `--tools` (restrict) and `--allowedTools` (pre-approve)
- [ ] I can rewrite vague prompt guidance into explicit, testable criteria
- [ ] I know few-shot teaches format and reasoning but cannot enforce ordering
- [ ] I know schemas fix syntax errors, review catches semantic errors
- [ ] I can design nullable/unclear/other schemas and a validation-retry prompt
- [ ] I know retries fix execution errors, never information gaps
- [ ] I know the full Batch vs real-time table and both trap directions
- [ ] I know why independent review sessions beat self-review
- [ ] I can explain lost-in-the-middle and its mitigations
- [ ] I can detect context degradation and remediate with scratchpads, compaction, and offloading
- [ ] I can design manifest-based crash recovery with idempotent tools
- [ ] I know why aggregate accuracy masks per-type failures and how stratified sampling fixes it
- [ ] I have the seven anti-patterns and five mental models down cold
- [ ] I have taken at least two full-length timed practice tests and reviewed every miss

Resources

Official (free):

- Anthropic Partner Academy: the exam guide, prep courses, and registration
- docs.anthropic.com: API, agents, tool use, MCP, Claude Code
- Anthropic Academy courses, mapped to this exam's domains:
  - Claude Code 101 and Claude Code in Action: the 20% Claude Code domain

- Introduction to Model Context Protocol and MCP: Advanced Topics: the 18% tool design and MCP domain
- Introduction to subagents and Introduction to agent skills: orchestration and Claude Code configuration
- Building with the Claude API: the API foundation everything else builds on

Free articles (preporato.com):

- CCA-F complete guide · Cheat sheet · Domains breakdown · Exam format · How to pass first attempt · Hardest topics by attempt data
- Technical guides: hooks · subagent orchestration · MCP tool design · MCP architecture · structured output · CLAUDE.md context management · permissions precedence

Practice:

- 30 practice questions with explanations in this repo, also as a timed practice test (free)
- The CCA-F page on Preporato: free sample questions to start, then the full tests below
- (Paid) Six full-length scenario-format practice tests and a 500-card flashcard deck built on the exact domain weights
- (Paid) 11 hands-on projects: build-and-submit work (agentic loops, hub-and-spoke systems, MCP servers, extraction pipelines) graded automatically against a rubric

---

Not affiliated with Anthropic. Exam details reflect the program as of October 2026; the Partner Academy is authoritative. Found an error? Open an issue.

Practice questions

30 questions for CCA-F, written for this guide in the exam's style and spread across its domains by the official weights. Questions that ask for more than one answer say how many. The answers and explanations follow in the next section.

The same questions run as a timed test with a score for every domain: practice test.

Scenario: Customer Support Escalation System

> A retailer's support system uses a coordinator agent that receives each customer conversation and delegates to three subagents: an order-lookup agent with read-only order tools, a refund agent that can issue refunds, and a policy agent that searches the returns-policy knowledge base. Company policy: refunds above $250 need a human's approval, and customers can come back to an open conversation within 7 days.

Question 1 *(Agentic Architecture & Orchestration)*

The refund agent needs the order total, the purchase date and the customer's stated reason to decide eligibility. What should the coordinator include when it delegates the task?

- **A.** An instruction for the refund agent to ask the customer to repeat the details
- **B.** The full raw transcript plus every prior tool output, so the refund agent misses nothing
- **C.** The specific facts the refund agent needs, written into the delegation prompt: order id, total, purchase date and the customer's stated reason
- **D.** Nothing extra, because subagents can read the coordinator's conversation history

Question 2 *(Context Management & Reliability)*

Which TWO events should trigger a handoff to a human agent? (Select TWO)

- **A.** Answering the customer's question takes three tool calls
- **B.** The customer asks to speak to a person
- **C.** The model reports a confidence score of 0.62 on its eligibility decision
- **D.** The customer writes in capital letters and says they are furious about a covered $90 refund
- **E.** The customer requests a $400 refund for an order the policy covers

Question 3 *(Tool Design & MCP Integration)*

The policy agent's knowledge-base search sometimes fails because the search service times out, and sometimes finds nothing because no policy covers the case. Both situations currently return an empty list. How should the tool respond instead?

- **A.** Keep returning an empty list in both cases and let the agent retry
- **B.** Return a structured error for the timeout (error type, retryable: true, what was attempted) and a successful empty result for the no-match case
- **C.** Return the closest-matching policy in both cases so the agent always has something to cite

- **D.** Raise an exception in both cases so the coordinator ends the conversation

Question 4 *(Agentic Architecture & Orchestration)*

A customer returns four days later to continue a conversation about a pending refund. Separately, a supervisor wants to test a different resolution on a copy of that conversation without changing what the customer sees. Which pairing fits?

- **A.** Start fresh sessions for both and paste the last assistant message into each system prompt
- **B.** Fork the session for the customer and resume it for the supervisor
- **C.** Resume the same session for both
- **D.** Resume the customer's session so the full history is restored, and fork a separate copy for the supervisor

Scenario: Multi-Repository Security Audit

> A security team runs a nightly audit across 40 repositories. A coordinator spawns one scanner subagent per repository. Each scanner has `Read`, `Grep` and `Glob` plus a dependency-check tool. The coordinator merges the findings into one report for the morning stand-up and opens tickets for new critical issues.

Question 5 *(Agentic Architecture & Orchestration)*

Which execution pattern fits the scanning stage and the report stage?

- **A.** Fan out the 40 scans in parallel, because they are independent, then run one merge step that builds the report from normalized results
- **B.** Run several report writers in parallel and concatenate their outputs
- **C.** Use one agent that scans all 40 repositories in a single context so it can spot cross-repository patterns
- **D.** Scan sequentially so each scanner can learn from the previous one's findings, then write the report in parallel sections

Question 6 *(Context Management & Reliability)*

Last night the audit crashed at repository 27. The rerun started again from repository 1 and opened duplicate tickets for findings already reported. Which TWO changes fix this? (Select TWO)

- **A.** Checkpoint progress to a manifest file (completed repositories, pending repositories, results so far) and resume from it after a crash
- **B.** Give the scanners larger context windows so they remember the previous run
- **C.** Run the audit weekly instead of nightly
- **D.** Make ticket creation idempotent, for example keyed by repository and finding id, so a replay can't create duplicates
- **E.** Ask the coordinator at startup which repositories it already finished

Question 7 *(Tool Design & MCP Integration)*

An engineer proposes giving every scanner the full tool set, including `Write`, `Bash` and the ticketing tool, 'in case they need it.' What is the best response?

- **A.** Replace the scanner tools with Bash alone, since Bash can do everything the other tools do
- **B.** Give the scanners every tool and add a system-prompt rule forbidding writes
- **C.** Keep each scanner on the read-only tools its job needs, and let only the coordinator open tickets from the merged findings
- **D.** Accept the proposal, because more tools make the scanners more capable

Question 8 *(Context Management & Reliability)*

The morning report said 'No critical issues found across all repositories.' The team later learns that six scanners hit their token budgets and skipped their largest files. What should the report format have required?

- **A.** Nothing; the scanners needed a more capable model
- **B.** A longer narrative summary from each scanner
- **C.** A confidence score from each scanner
- **D.** A coverage annotation from every scanner (files reviewed, files skipped with reasons, coverage counts) carried into the merged report

Standalone questions

Question 9 *(Agentic Architecture & Orchestration)*

After the first tool call, every follow-up request in your agent loop fails with a 400 error. Here is how the loop builds the next request. What is wrong?

```
response = client.messages.create(model=MODEL, max_tokens=1024,
                                  tools=tools, messages=messages)
if response.stop_reason == "tool_use":
    call = next(b for b in response.content if b.type == "tool_use")
    result = run_tool(call.name, call.input)
    messages.append({"role": "user", "content": [{
        "type": "tool_result",
        "tool_use_id": call.id,
        "content": result,
    }]})
```

- **A.** The loop should set `tool_choice` to `any` after the first call
- **B.** The assistant turn containing the `tool_use` block is never appended, so the `tool_result` refers to a call the next request doesn't contain
- **C.** `tool_result` blocks belong in the system prompt
- **D.** `max_tokens` is too low for a tool loop

Question 10 *(Agentic Architecture & Orchestration)*

Your research agent uses the server-side web search tool. Some responses come back with `stop_reason` `pause_turn`, and the loop treats them as finished, returning half-done research. What is the correct handling?

- **A.** Send a follow-up request with the paused assistant response appended as-is, so Claude can continue the server-side work
- **B.** Raise `max_tokens` and send the original request again
- **C.** Treat `pause_turn` the same as `end_turn`
- **D.** Run the web search locally and return a `tool_result` for it

Question 11 *(Agentic Architecture & Orchestration)*

A team answers one-line FAQ questions from a two-page policy with a six-subagent pipeline: retriever, reader, drafter, tone checker, fact checker and formatter. Answers take 25 seconds and cost far more than expected. What is the best change?

- **A.** Collapse it to a single model call with the policy in context, plus a lightweight validation step if needed
- **B.** Move every subagent to the most capable model to cut retries
- **C.** Run the six subagents in parallel under a coordinator
- **D.** Cache each subagent's output

Question 12 *(Agentic Architecture & Orchestration)*

A payments service's error rate spiked. The cause could be in code, configuration, infrastructure or a third-party API, and what to check next depends on what each check finds. How should the coordinator decompose the investigation?

- **A.** One prompt asking Claude for the root cause
- **B.** A fixed pipeline that runs the four checks in a set order
- **C.** Dynamic decomposition: plan the next subtasks from each finding, with a token budget and iteration cap enforced in code as safety nets
- **D.** Run every conceivable check in parallel with no coordinator

Question 13 *(Agentic Architecture & Orchestration)*

Long research tasks sometimes use ten times the expected tokens. Which TWO belong in a reliable token-budget design? (Select TWO)

- **A.** Setting `max_tokens` to 50 on every call
- **B.** Asking Claude each turn how many tokens it has used so far
- **C.** A system-prompt instruction asking Claude to stay under 50,000 tokens
- **D.** Track cumulative input and output tokens from each response's usage field inside the loop
- **E.** Stop the loop in code when the budget is reached, with a wrap-up instruction injected at about 80% of the budget

Question 14 *(Tool Design & MCP Integration)*

A team's MCP server should (1) let Claude create Jira tickets, (2) expose the team's API schema files so a user can attach them as context, and (3) provide a reusable 'triage this incident' template with the team's severity rubric. Which MCP primitives fit, in that order?

- **A.** Tool, tool, tool
- **B.** Resource, tool, prompt
- **C.** Prompt, resource, tool
- **D.** Tool, resource, prompt

Question 15 *(Tool Design & MCP Integration)*

Every developer on a team should get the same internal MCP server in Claude Code, and the server needs an API token. Which TWO choices are correct? (Select TWO)

- **A.** Put the token in `CLAUDE.md` so Claude can pass it to the server
- **B.** Paste the raw token into `.mcp.json` so setup is a single step
- **C.** Define the server in the project's `.mcp.json` and commit it to the repository
- **D.** Reference the token in `.mcp.json` as `${INTERNAL_API_TOKEN}` and have each developer set that environment variable
- **E.** Have each developer add the server with local scope by following a wiki page

Question 16 *(Tool Design & MCP Integration)*

When users ask to change their email address, Claude often calls `get_customer` instead of `update_customer`, then tells the user the change is done. Both tools exist. What is the first fix?

- **A.** Add a system-prompt line telling Claude to use `update_customer` for updates
- **B.** Rewrite both descriptions: what each tool does, when to use it, and when to use the other one instead, with `get_customer` described as read-only
- **C.** Rename `get_customer` to `fetch_customer`
- **D.** Remove `get_customer` from the agent

Question 17 *(Claude Code Configuration & Workflows)*

Your team wants every file Claude edits to be formatted with Prettier immediately after each edit, for everyone working in the repository. Which configuration does that?

- **A.** A hook in each developer's `~/.claude/settings.json`
- **B.** A `CLAUDE.md` line: 'Run Prettier after every edit'
- **C.** A `PostToolUse` hook in the committed `.claude/settings.json`, matching `Edit|Write`, that runs Prettier on the edited file
- **D.** A `PreToolUse` hook on `Edit` that runs Prettier

Question 18 *(Claude Code Configuration & Workflows)*

A team requires that Claude never ends its work while the test suite is failing. Which hook enforces this?

- **A.** A `SessionEnd` hook that runs the tests

- **B.** A `Stop` hook that runs the tests and exits with code 2, printing the failures to stderr, when any test fails
- **C.** A `PostToolUse` hook that runs the tests after every file read
- **D.** A `UserPromptSubmit` hook that runs the tests before each prompt

Question 19 *(Claude Code Configuration & Workflows)*

A `PostToolUse` hook on `Write` scans each file Claude writes. It finds an API key in a file Claude just wrote and exits with code 2. What happens?

- **A.** The file has already been written; Claude receives the hook's stderr message and can act on it, for example by removing the key
- **B.** Claude Code shuts down the session
- **C.** Nothing happens, because exit code 2 is ignored for `PostToolUse`
- **D.** The write is cancelled and the file is never created

Question 20 *(Claude Code Configuration & Workflows)*

A developer asks Claude Code to migrate 40 files from a callback-based API client to async/await. The change touches shared types, and there are two reasonable migration strategies. What is the best way to start?

- **A.** One prompt for the whole change, then review the final diff
- **B.** Direct execution, file by file, fixing problems as they appear
- **C.** A `CLAUDE.md` rule saying 'use async/await'
- **D.** Plan Mode: let Claude explore the codebase and propose a migration plan to review before any file changes

Question 21 *(Claude Code Configuration & Workflows)*

A team's dependency-audit skill prints thousands of lines of output, and teammates complain it floods the main conversation. Which frontmatter change fixes this?

- **A.** `context: fork`, so the skill runs in an isolated subagent and returns its result
- **B.** `disable-model-invocation: true`
- **C.** `allowed-tools: Read`
- **D.** `model: haiku`

Question 22 *(Claude Code Configuration & Workflows)*

In a monorepo, files under `services/payments/` follow strict rules (all amounts in integer cents, no floats). The rules confuse Claude when it works on the frontend. Which TWO placements load these rules only when Claude works in the payments code? (Select TWO)

- **A.** A section titled 'Payments only' in the root `CLAUDE.md`
- **B.** A `CLAUDE.md` file inside `services/payments/`
- **C.** A file in `.claude/rules/` with `paths: ["services/payments/**"]` in its frontmatter
- **D.** Each developer's `~/.claude/CLAUDE.md`

- **E.** The project's `.mcp.json`

Question 23 *(Prompt Engineering & Structured Output)*

A code-review prompt says 'Be thorough but avoid nitpicks.' Reviews flag style issues inconsistently and miss SQL injection. Which rewrite fixes the cause?

- **A.** 'You are a world-class senior security engineer.'
- **B.** 'Be extremely thorough and never miss anything.'
- **C.** 'Be conservative with your comments.'
- **D.** 'Report only these issues: SQL injection, XSS, authentication bypass and secrets in code. Report high or critical severity only. Do not comment on formatting or naming.'

Question 24 *(Prompt Engineering & Structured Output)*

A classifier should flag support tickets that mention a physical safety risk. In testing it flags every ticket containing the word 'fire', including 'firewall rules reset after update.' What is the best few-shot improvement?

- **A.** An instruction: 'Don't be fooled by keywords.'
- **B.** Two to four examples that include the firewall ticket labeled not-safety with a one-line reason, a genuine safety ticket, and an ambiguous case
- **C.** Twenty examples of genuine safety tickets
- **D.** A lower temperature

Question 25 *(Prompt Engineering & Structured Output)*

Your team plans to use structured outputs (a JSON schema in `output_config.format`) for invoice extraction. Which TWO statements about this feature are true? (Select TWO)

- **A.** It guarantees the extracted values are correct for the source document
- **B.** It works only when `tool_choice` forces a specific tool
- **C.** It removes the need to validate business rules, such as totals matching line items
- **D.** Constrained decoding makes the response valid JSON that matches the schema's structure and types
- **E.** A refusal or a `max_tokens` cutoff can still produce output that doesn't match the schema, so check `stop_reason`

Question 26 *(Prompt Engineering & Structured Output)*

An extraction fails validation because `invoice_date` came back as 2026-04-31, a date that doesn't exist. What should the retry request contain?

- **A.** The failed output and the words 'try again'
- **B.** The original document again with no other changes
- **C.** The original document, the failed output, and the specific error ('`invoice_date` 2026-04-31 is not a valid date'), with an instruction to correct only that field
- **D.** The same request sent to a more capable model

Question 27 *(Prompt Engineering & Structured Output)*

A product team wants to move its live chat assistant to the Message Batches API to cut costs in half. What is the right assessment?

- **A.** Reject it, because batch requests can't include a system prompt
- **B.** Approve it: the same quality at half the price
- **C.** Approve it, provided the app polls for results every second
- **D.** Reject it: batches carry no latency guarantee (most finish within an hour, some take up to 24), so a waiting user can't be served; cut costs with prompt caching or by routing routine messages to a cheaper tier

Question 28 *(Prompt Engineering & Structured Output)*

One prompt asks Claude to extract 30 fields from a contract, normalize every currency, classify the risk clauses and write an executive summary. Results vary from run to run, and errors are hard to trace. What is the best restructuring?

- **A.** A chain of focused prompts (extract, validate, normalize, classify, summarize), each with its own instructions and checks
- **B.** The same single prompt on a more capable model
- **C.** Ask for the executive summary first, then the fields
- **D.** The same single prompt with each requirement marked IMPORTANT

Question 29 *(Context Management & Reliability)*

A knowledge-base Q&A agent receives 40 retrieved passages per question. The passage with the answer usually lands near the middle, and answers often miss it. Which TWO changes address this? (Select TWO)

- **A.** Retrieve 80 passages instead of 40
- **B.** Re-rank the passages and pass only the most relevant few
- **C.** Raise the temperature
- **D.** Put the passages first and the question and instructions at the end of the prompt
- **E.** Switch to a model with a larger context window

Question 30 *(Context Management & Reliability)*

Two internal documents disagree about the warranty period: a 2024 policy says 12 months and a 2026 update says 24 months. What should the agent's answer do?

- **A.** Decline to answer until someone fixes the documents
- **B.** State 24 months without mentioning the conflict, because the newer document wins
- **C.** Give the answer with attribution: both values, their documents and dates, the newer one identified as likely current, and the conflict flagged for review
- **D.** Average the two and state 18 months

Answers and explanations

Question 1: C. Subagents in a hub-and-spoke design start with their own context window and see only what the coordinator passes to them, so C is the working design: the minimum necessary context, stated explicitly. D assumes an inheritance that doesn't exist. B spends tokens and attention on material the decision doesn't need, which is where lost-in-the-middle errors start. A makes the customer pay for a gap the coordinator could close with facts it already has.

Question 2: B and E. An explicit request for a human (B) is a standing escalation trigger, and the $400 refund (E) crosses the $250 approval threshold, which makes the handoff mandatory under policy. Sentiment alone (D) is no trigger when the request sits inside clear policy; the agent can resolve it while acknowledging the frustration. Self-reported confidence (C) is poorly calibrated and makes an unreliable trigger. Task complexity (A) is the agent's job.

Question 3: B. B separates a transient failure from a valid empty result, and the two need different recovery: retry the search, or tell the customer no policy applies. With A, the agent can't tell an outage from a real answer and may report that no policy covers the case during an outage. D ends conversations over a recoverable timeout. C invents relevance, which leads the agent to cite a policy that doesn't apply.

Question 4: D. Resuming continues a session with its history intact, which is what the returning customer needs. Forking creates an independent branch from the same history, and changes in the fork never flow back, which is what the supervisor needs. D matches both. B swaps the two. C lets the supervisor's experiment change the customer's conversation. A throws away most of the context and relies on one message to carry it.

Question 5: A. Independent work runs in parallel and dependent work runs in sequence, so the hybrid in A matches the data dependencies: the scans don't need each other, and the report needs all of them. D serializes independent work and slows the audit for no gain. C pushes 40 repositories into one context, which degrades attention and turns one failure into a total failure. B produces a report with no single view of the findings, which is what the merge step exists for.

Question 6: A and D. A manifest (A) lets the restarted coordinator continue from repository 27, and idempotent side effects (D) make any replayed step harmless; together they turn a crash into a delay. Context windows (B) don't survive a process crash, and the coordinator has no memory of the previous run to ask about (E), because the API is stateless. Running less often (C) leaves the failure in place.

Question 7: C. Least privilege scopes each agent to what its responsibility requires. C also keeps tool selection accurate and stops a misbehaving scanner from changing code or opening tickets. D widens the blast radius and adds selection noise for capabilities nobody uses. B relies on a probabilistic instruction to restrain a capability that should never be there. A hands the scanners the broadest tool of all.

Question 8: D. Without explicit coverage, 'no issues found' can't be told apart from 'no issues found in the part that ran.' D makes the gap visible so the skipped files can be queued for a focused pass. Self-reported confidence (C) is poorly calibrated and says nothing about what was skipped. A longer summary (B) adds words without the counts. A stronger model (A) can still hit a budget, and the report would still hide it.

Question 9: B. The API is stateless, so each request must carry the whole exchange: the assistant message with the `tool_use` block comes first, then the user message with the matching `tool_result` (B). Without it, the `tool_use_id` points at nothing. C misplaces tool results, which go in a user turn. D and A change settings that have nothing to do with a malformed history.

Question 10: A. `pause_turn` means a server-side tool loop reached its iteration limit mid-task. The documented handling is A: return the response as the latest assistant turn, and Claude picks up where it stopped. C is the bug in the question. D treats a server tool like a client tool; there is no `tool_use` for your code to answer. B restarts the work and pays for it twice.

Question 11: A. This is over-decomposition: the task is small, single-domain and fits in one context, so the subagents add orchestration cost without any isolation benefit, and A removes the overhead. Parallelizing (C) doesn't fit a chain where each step needs the previous step's output. B raises the cost of a design that shouldn't exist. D helps only with repeated identical questions and leaves the latency of the chain in place.

Question 12: C. Open-ended investigations, where the next step depends on what you just learned, call for dynamic decomposition. Because that pattern costs more and can wander, C pairs it with limits enforced outside the model. A fixed pipeline (B) suits workflows that are known in advance. D wastes effort and leaves nobody to connect the findings. A asks for an answer without giving the model any way to gather evidence.

Question 13: D and E. Budgets are enforced from outside the model: measure from the usage field (D) and act on the measurement with a soft warning and a hard stop (E). An instruction (C) and the model's own estimate (B) are probabilistic and unmeasured. A tiny `max_tokens` (A) truncates every response, so turns end at the cap before they finish.

Question 14: D. Creating a ticket is an action, so it's a tool. Schema files are read-only data for the client to browse and attach, so they're a resource. The packaged triage template is a prompt. D matches action, data and know-how in that order. A turns data and a template into callable functions. B and C put the action in the wrong primitive.

Question 15: C and D. Project scope (C) is the configuration that reaches the whole team through version control, and environment variable expansion (D) keeps the secret out of the repository. B commits a credential. E relies on every developer repeating a manual step, and local scope never propagates. A puts a secret into the model's context, where it can leak through outputs or logs.

Question 16: B. Claude selects tools mainly from their descriptions, so overlapping or vague descriptions are the first thing to fix (B), and the boundary statements ('to change a record, use `update_customer`') are what resolve this confusion. A rename (C) changes little while the descriptions stay vague. A system-prompt patch (A) is weaker than fixing the routing signal itself. Removing the read tool (D) breaks every legitimate lookup.

Question 17: C. Formatting has to run after the file changes, which is `PostToolUse`, and the committed project settings file shares the hook with the whole team, so C covers both requirements. D runs before the edit exists. B is guidance Claude may skip, while a hook runs every time. A works only for developers who set it up themselves.

Question 18: B. Exit code 2 from a `Stop` hook prevents Claude from stopping and feeds the stderr text back, so Claude keeps working on the failures (B). `SessionEnd` (A) fires as the session closes and can't block anything. D tests before the work happens. C runs constantly at the wrong moment and can't stop Claude from finishing.

Question 19: A. `PostToolUse` fires after the tool has run, so it can't undo the write; exit code 2 shows its stderr to Claude (A). To stop the write itself, the check belongs in a `PreToolUse` hook, where exit code 2 blocks the call, which is the behavior D describes. B and C describe behaviors that don't exist.

Question 20: D. Multi-file, cross-cutting changes with more than one valid approach are Plan Mode's case (D): the strategy gets agreed before 40 files move. B and A commit to an approach before anyone has compared the options, and a wrong choice is expensive to unwind. C states a preference without planning the migration.

Question 21: A. `context: fork` (A) runs the skill in its own subagent context, so the verbose work stays out of the main conversation. B only stops Claude from invoking the skill automatically; the output still floods the conversation whenever someone runs it. C changes which tools are pre-approved and D changes the model, and neither moves the output.

Question 22: B and C. A path-scoped rule (C) loads when Claude reads or edits a matching file, and a subdirectory `CLAUDE.md` (B) loads on demand when Claude works with files in that directory. The root `CLAUDE.md` (A) loads in every session, frontend work included. D is personal and reaches nobody else. E configures MCP servers and holds no instructions.

Question 23: D. Vague instructions produce inconsistent behavior because the model has to guess the threshold. D names the categories, the severity floor and the exclusions, and anyone reading a review can check whether it followed them. B and C push the same vague guidance in two directions. A role line (A) without criteria leaves the same guesswork.

Question 24: B. Few-shot examples teach reasoning and edge cases best when they include something the model should leave alone, so the tricky negative in B is the example that matters. C piles on positives and teaches nothing about the false positives. A is vague guidance. Temperature (D) changes randomness and leaves the misunderstanding in place.

Question 25: D and E. Structured outputs guarantee format: valid, schema-compliant JSON (D), with the documented exceptions of refusals and truncation (E). Format says nothing about whether the values are right, so semantic checks such as totals and dates stay in your pipeline, which rules out A and C. B confuses the feature with forced tool use; structured outputs are requested through `output_config.format`.

Question 26: C. A useful retry tells the model exactly what failed and gives it the source to fix it against (C). B repeats the conditions that produced the error. A removes the source the model needs to correct the date. D changes the model without telling it what went wrong.

Question 27: D. The batch discount comes with asynchronous processing and no latency guarantee, which rules it out for anyone waiting on an answer (D). B ignores the latency. Polling (C) doesn't speed up processing. A is false: batch requests take the same parameters as regular Messages requests.

Question 28: A. Prompt chaining gives each stage one job and a validation point, so a failure shows up in the stage that caused it (A). Emphasis (D) doesn't reduce the number of jobs in one prompt. A stronger model (B) may help a little and leaves errors untraceable. Reordering (C) builds the summary on fields nobody has checked yet.

Question 29: B and D. Attention is weakest in the middle of long contexts, so the fixes shrink the middle and move what matters to the edges: fewer, better passages (B), and long material first with the question last, which Anthropic's long-context guidance recommends (D). A bigger window (E) and more passages (A) make the middle longer. Temperature (C) affects randomness and does nothing for attention.

Question 30: C. Provenance keeps each claim tied to its source and surfaces conflicts instead of settling them silently (C). Silently picking one value (B) is the tested anti-pattern even when the choice happens to be right, because nobody learns that the documents disagree. D produces a number neither source supports. A withholds an answer the agent can give with a caveat.

Keep going

- **Full-length practice.** The CCA-F page on Preporato has free sample questions and six full-length timed practice tests (paid) in the exam's format, with a score for every domain.
- **Updates and corrections.** The latest edition, the practice test and the other three guides are at github.com/preporato/claude-certification-guide.